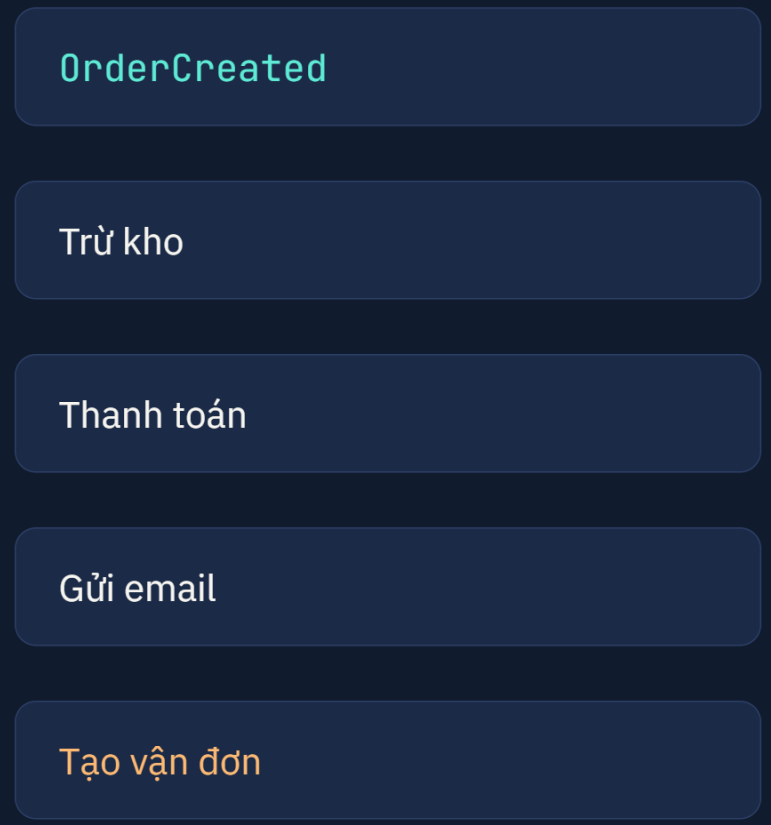
KIẾN TRÚC PHẦN MỀM · THI GIỮA KỲ

Event-Driven Architecture (EDA)

Công bố sự việc đã xảy ra thay vì gọi nhau trực tiếp



VẤN ĐỀ

Order Service gọi trực tiếp từng service bằng REST

Phải biết mọi service

Kho, thanh toán, email, vận đơn: tất cả đều phải được Order Service biết.

Phải chờ từng cái

Mỗi lời gọi phải đợi service kia trả lời.

Một service chết

Cả đơn hàng thất bại.

Thêm chức năng

Phải sửa lại Order Service.

Đây là coupling chặt của kiến trúc hướng yêu cầu (request-driven).

EDA LÀ GÌ

Công bố sự kiện thay vì ra lệnh

`OrderCreated` Service chỉ công bố sự kiện, không biết ai sẽ xử lý.

	Event	Command
Ý nghĩa	Sự thật đã xảy ra	Yêu cầu “hãy làm việc này”
Tính chất	Thì quá khứ, bất biến, không thể sửa	Mệnh lệnh
Người nhận	Consumer nào đăng ký đều nhận	Một bên cụ thể

BA VAI TRÒ CHÍNH

Producer, event broker và consumer

Producer

Nơi phát sinh và công bố sự kiện.

Event broker

Kênh trung gian nhận, lưu và phân phối sự kiện. Ví dụ: Kafka, RabbitMQ.

Consumer

Đăng ký nhận sự kiện và tự quyết định phản ứng.

Producer không biết có bao nhiêu consumer, và cũng không chờ họ.

HAI TOPOLOGY

Choreography và orchestration

Broker topology

Không có bộ điều phối trung tâm. Mỗi service nghe sự kiện và tự xử lý, như các vũ công tự phối hợp.

Ưu: rất lỏng lẻo và linh hoạt.

Nhược: khó nhìn ra toàn bộ luồng nghiệp vụ.

Mediator topology

Một orchestrator điều phối thứ tự các bước và xử lý lỗi.

Ưu: luồng rõ ràng, dễ kiểm soát hơn.

Nhược: orchestrator thành điểm tập trung, coupling cao hơn.

CÁC MẪU SỰ KIỆN

Ba cách dùng sự kiện phổ biến

Event notification

Chỉ báo có chuyện xảy ra. Consumer muốn biết thêm thì tự hỏi lại.

Event-carried state transfer

Sự kiện mang đủ dữ liệu, consumer không cần gọi ngược lại producer.

Event sourcing

Sự kiện chính là nguồn lưu trữ trạng thái. Đây là chủ đề 5.

KIỂU PHÂN PHỐI

Publish/subscribe: một sự kiện đến nhiều consumer.

Queue: chia công việc cho một trong nhiều worker.

LỢI ÍCH

Lỏng lẻo, dễ mở rộng, chịu lỗi tốt

Loose coupling

Service độc lập, chỉ thống nhất với nhau về định dạng sự kiện.

Dễ mở rộng chức năng

Chỉ cần thêm một consumer mới, không sửa producer.

Khả năng chịu lỗi

Broker giữ sự kiện, consumer sống lại vẫn xử lý tiếp, không mất dữ liệu.

Scale độc lập, bất đồng bộ

Hệ thống chịu được tải đột biến.

Phù hợp với xử lý thời gian thực và tích hợp nhiều hệ thống.

THÁCH THỨC 1/2

Nhất quán, trùng lặp và thứ tự sự kiện

Thách thức	Vấn đề	Cách xử lý
Eventual consistency	Dữ liệu giữa các service chỉ nhất quán sau một khoảng trễ	Saga, với bước bù trừ khi lỗi
Trùng lặp sự kiện	Broker thường đảm bảo at-least-once, sự kiện có thể đến hai lần	Consumer idempotent
Thứ tự sự kiện	Thường chỉ đảm bảo trong cùng partition	Partition key hợp lý, ví dụ orderId

THÁCH THỨC 2/2

Lỗi, vận hành và định dạng sự kiện

Thách thức	Vấn đề	Cách xử lý
Mất sự kiện, lỗi xử lý	Sự kiện không xử lý được	Retry và Dead Letter Queue
Ghi DB và phát sự kiện	Hai việc có thể lệch nhau	Outbox pattern
Khó debug	Luồng bất đồng bộ khó lần vết	Correlation ID, distributed tracing
Đổi định dạng sự kiện	Có thể phá consumer cũ	Quản lý schema và versioning

ÁP DỤNG

Khi nào nên và không nên dùng EDA

Nên dùng

- Nhiều thành phần cần phản ứng với cùng một sự kiện
- Cần scale độc lập
- Tích hợp bất đồng bộ

Không nên dùng

- Hệ thống nhỏ
- Nghiệp vụ cần phản hồi đồng bộ và nhất quán tức thời
- Đội ngũ chưa sẵn sàng vận hành broker và xử lý các vấn đề trên